3. Summary

TravelNotes is a personal travel-tracking app. It allows users to make notes about visited cities and places, and probably rate them. It holds a hierarchy of “Countries → Cities → Places”, each marked as “Visited” or “Planned”, and lets the user attach their own notes and reviews to any level of that hierarchy. The list of countries, cities is pre-defined before the user input and stored in the database. The places and notes are created by the user and saved into the database.

Each place has 4 data points: name, status, map coordinates (google maps api) and a category (landmark, museum, cafe, etc.). A user note is a text entry (title, rating, date visited, free text) attached to a place

4. Introduction

TravelNotes consist of 3 pages:

1. Register
2. Login
3. Dashboard. That's the main data page.

Progress been made:

1. Created barebones Express MVC skeleton in app_server/ with routes, controllers, and views folders
2. Created three static pages (Register, Login, Dashboard) each rendering a page-specific title via res.render()
3. Added styles using Bootstrap elements. Shared elements (footer) in layout.pug, login and register cards for corresponding pages.
4. Added dummy (hardcoded) data for each page
5.

GitHub repository:

https://github.com/Danil-Kramnov/travelnotes

Render URL:

xxx

5. Views

Three Pug views currently exist, all extending a shared layout:

- app_server/views/layout.pug – shared head/body + footer block.
- app_server/views/register.pug – Register page + registration card.
- app_server/views/login.pug – Login page + login card.
- app_server/views/dashboard.pug – Dashboard with dummy data.

layout.pug

```
doctype html

html
```

```
 head
   meta(name='viewport', content='width=device-width, initial-scale=1.0')
   title= title
   //- Footer design from https://bootsnipp.com/snippets/lGEO2
   link(rel='stylesheet',
href='https://maxcdn.bootstrapcdn.com/bootstrap/4.1.1/css/bootstrap.min.css')
   link(rel='stylesheet', href='/stylesheets/style.css')


   //- All html to pug conversion done by this: https://html-to-pug.com/
 body.d-flex.flex-column.min-vh-100
   block navbar


   main.flex-grow-1
     block content


   footer.bottom.section-padding.bg-dark
     .container
       .row
         .col-md-12.text-center
           .copyright
             p © #{new Date().getFullYear()} TravelNotes — All rights reserved.


   script(src='https://cdnjs.cloudflare.com/ajax/libs/jquery/3.2.1/jquery.min.js')
   script(src='https://maxcdn.bootstrapcdn.com/bootstrap/4.1.1/js/bootstrap.min.js')
```

layout .pug contains shared footer and styles across all pages

register.pug (login.pug follow the same pattern)

```
extends layout

block content
```

```
//- Register card from: https://bootsnipp.com/snippets/z812X
.container
  .row
    .col-md-5.mx-auto
      .myform
        .logo.mb-3
          .col-md-12.text-center
            h1 Sign up
        form(action='/register', method='POST', name='registration')
          .form-group
            label(for='name') Name
            input#name.form-control(type='text', name='name', value=user.name,
placeholder='Enter your name')
          .form-group
            label(for='email') Email address
            input#email.form-control(type='email', name='email', value=user.email,
placeholder='Enter email')
          .form-group
            label(for='password') Password
            input#password.form-control(type='password', name='password',
placeholder='Enter password')
          .form-group
            label(for='confirmPassword') Confirm password
            input#confirmPassword.form-control(type='password',
name='confirmPassword', placeholder='Confirm password')
          .col-md-12.text-center.mb-3
            button.btn.btn-block.mybtn.btn-primary.tx-tfm(type='submit') Register
          .form-group
            p.text-center
```

```
            a(href='/login') Already have an account?
```

Each page renders its own title, user.name and user.email passed in from its controller via res.render()

Dashboard.pug

```
extends layout

//- Bootstrap components used: navbar, card, list-group, badge, form-row grid
(https://getbootstrap.com/docs/4.1/components/navbar/#how-it-works)

block navbar

 nav.navbar.navbar-expand-lg.navbar-dark.bg-dark

   a.navbar-brand(href='/dashboard') TravelNotes

   button.navbar-toggler(type='button', data-toggle='collapse',
data-target='#navbarContent', aria-controls='navbarContent', aria-expanded='false',
aria-label='Toggle navigation')

     span.navbar-toggler-icon

   #navbarContent.collapse.navbar-collapse.justify-content-end

     span.navbar-text.text-white.mr-3 Hi, Daniel

     a.btn.btn-outline-light.btn-sm(href='/login') Logout



block content

   //- Card visual styling: https://bootsnipp.com/snippets/67z63



   .container.mt-4

   h2.mb-3 My Destinations



   each country in countries

     .card.mb-3

       .card-header

         strong= country.name

       .card-body
```

```
        each city in country.cities
          h5.mt-2= city.name
          ul.list-group.mb-3
            each place in city.places

li.list-group-item.d-flex.flex-column.flex-sm-row.justify-content-between.align-items-start
                div
                  div
                    strong= place.name
                    small.text-muted= ' · ' + place.category
                  if place.note
                    div.text-muted= place.note
                  else
                    div.text-muted.font-italic no note yet
                span.badge(class=place.status === 'Visited' ? 'badge-primary' : 'badge-warning')= place.status


  .card.mt-4
    .card-body
      h5.card-title Add a note
      form(action='/notes', method='POST')
        .form-row
          .form-group.col-md-6
            label(for='noteTitle') Title
            input#noteTitle.form-control(type='text', name='title', placeholder='Title')
          .form-group.col-md-6
            label(for='noteRating') Rating (1-5)
```

```
            input#noteRating.form-control(type='number', name='rating', min='1',
max='5', placeholder='e.g. 4')

          .form-group

            label(for='noteText') Notes

            textarea#noteText.form-control(name='text', rows='3', placeholder='Write
your note...')

          button.btn.btn-success(type='submit') Save note
```

Dashboard page contains navbar, row-grid, list-group from bootstrap docs. For the main card I've used another custom bootsnip element (referenced in the code)

UI framework (Bootstrap)

On all my pages I've been using custom bootsnip cards which I referenced throughout the code. And for dashboard I used some atomic elements from bootstrap 4.1 documentation (also referenced)

Data binding

Each controller builds a plain JavaScript object:

```
  const countries = [

{

    name: 'Ireland',

    cities: [

      {

        name: 'Tralee',

        places: [

          {

            name: 'Blennerville Windmill',

            category: 'Landmark',

            status: 'Visited',

            note: 'Great little museum, windy day',

          },

…
```

and passes it as the second argument to res.render(), which the corresponding Pug view then reads:

```
res.render('dashboard', { title: 'Dashboard', countries });
```

The view consumes it with Pug's each loop and attribute interpolation:

```
each country in countries

    .card.mb-3

      .card-header

        strong= country.name
```

At this stage the data (countries array, user object for login/register) is hardcoded inside the controller

Responsive design

12-column grid used for layout structure in a few places:

```
footer.bottom.section-padding.bg-dark

   .container

      .row

        .col-md-12.text-center
```

The login/register forms center a card using an offset-style grid:

```
.container

  .row

    .col-md-5.mx-auto

      .myform

        .logo.mb-3

          .col-md-12.text-center

            h1 Sign up
```

This is full-width (12 columns) below the md breakpoint, then narrows to 5 columns and centers itself once the viewport is wide enough. The dashboard's note-entry form places two fields side by side on larger screens using form-row.

Each field takes 6 of 12 columns (half-width) at md and above, stacking to full width below it.

```
.form-group.col-md-6

            label(for='noteTitle') Title

            input#noteTitle.form-control(type='text', name='title',
placeholder='Title')

          .form-group.col-md-6

            label(for='noteRating') Rating (1-5)
```

navbar-toggler hides the nav links behind a hamburger button below the lg:

```
button.navbar-toggler(type='button', data-toggle='collapse',
data-target='#navbarContent', aria-controls='navbarContent', aria-expanded='false',
aria-label='Toggle navigation')

    span.navbar-toggler-icon
```

6. Controllers

One controller per page, each exporting an index function used by the matching route:

app_server/controllers/registerController.js

```
const index = function (req, res) {

 const user = {

   name: 'test',

   email: 'test@example.com',

 };



 res.render('register', { title: 'Register', user });

};

module.exports = { index, };
```

loginController.js and registerController.js follow the same shape, rendering 'login' and 'registration' respectively.

app_server/controllers/dashboardController.js

dashboardController.js contains dummy data

```
const index = function (req, res) {

 const countries = [

   {

     name: 'Ireland',

     cities: [

       {

         name: 'Tralee',

         places: [

           {

             name: 'Blennerville Windmill',

             category: 'Landmark',

             status: 'Visited',

             note: 'Great little museum, windy day',

           },

           {

             name: 'Siamsa Tíre',

             category: 'Theatre',

             status: 'Planned',

             note: null,

           },

         ],

       },

       {

         name: 'Dublin',
```

```
        places: [
          {
            name: 'Trinity College Library',
            category: 'Landmark',
            status: 'Planned',
            note: null,
          },
          {
            name: 'Guinness Storehouse',
            category: 'Museum',
            status: 'Visited',
            note: 'Busy but worth it for the rooftop view',
          },
        ],
      },
    ],
  },
];

 res.render('dashboard', { title: 'Dashboard', countries });
};

module.exports = { index, };
```

Data structure in the controller

xxx

7. Routes

app_server/routes/index.js maps each URL to its controller:

```
const express = require('express');
```

```
const router = express.Router();

const ctrlRegister = require('../controllers/registerController');

const ctrlLogin = require('../controllers/loginController');

const ctrlDashboard = require('../controllers/dashboardController');



router.get('/register', ctrlRegister.index);

router.get('/login', ctrlLogin.index);

router.get('/dashboard', ctrlDashboard.index);

router.get('/', (req, res) => res.redirect('/login'));



router.post('/login', (req, res) => res.redirect('/dashboard'));

router.post('/register', (req, res) => res.redirect('/dashboard'));



module.exports = router;
```

'/' currently redirects straight to '/login' or to /dashboard after valid login/registration with dummy data

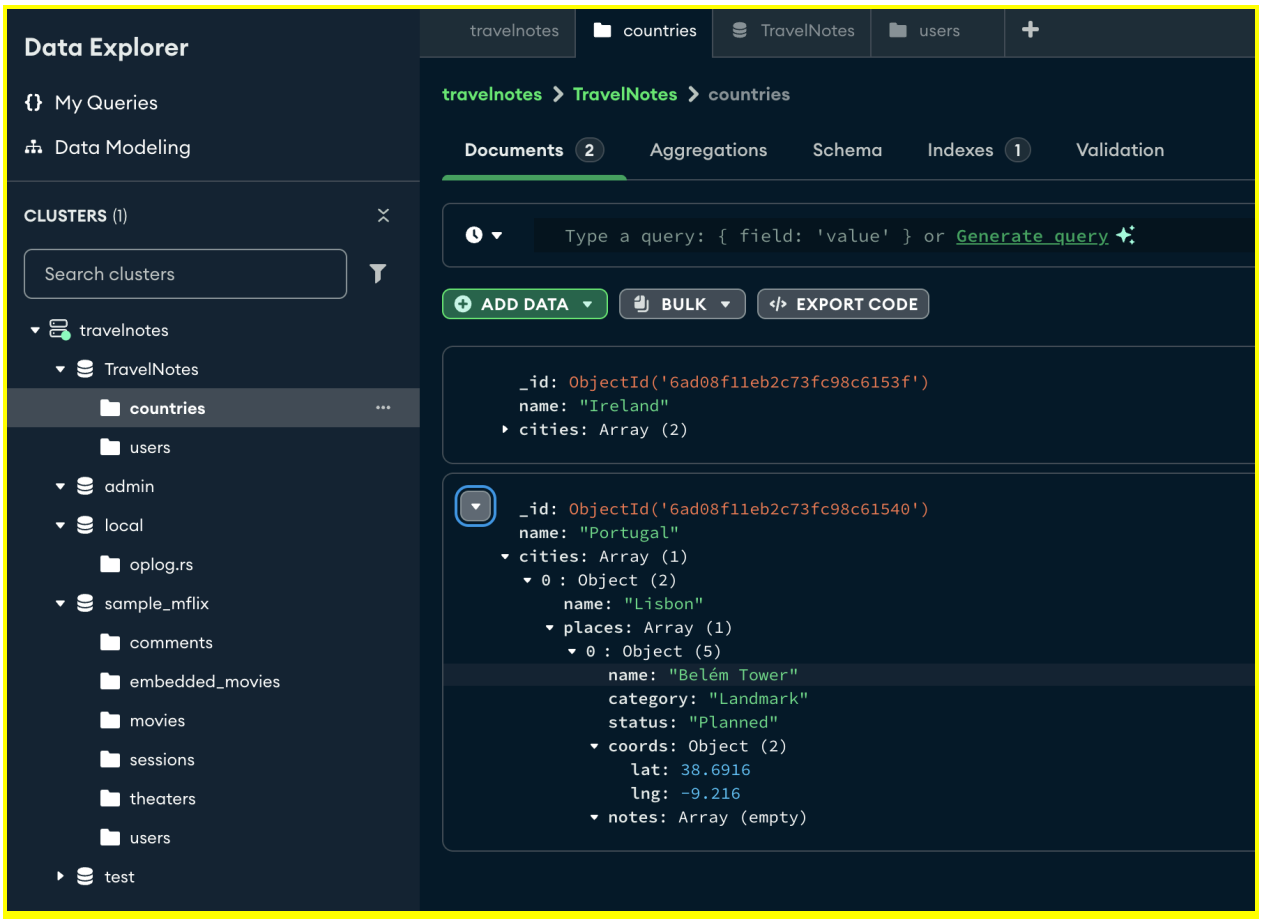
xxx Do I need to update this section later after I have the data wired up?

8. Model

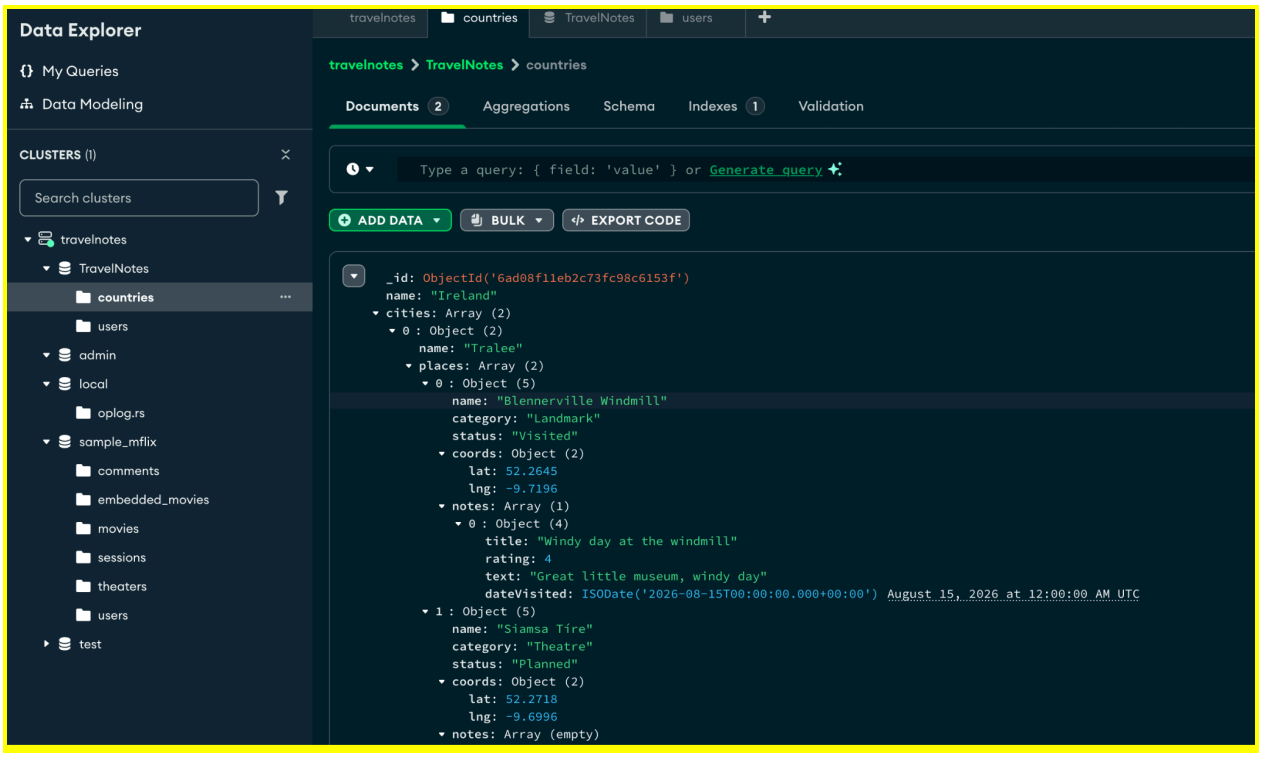
coords - variables for latitude and longitude (lat/lng)

Schema validation: for the status I've used enum, so only "Visited" or "Planned" can be saved.

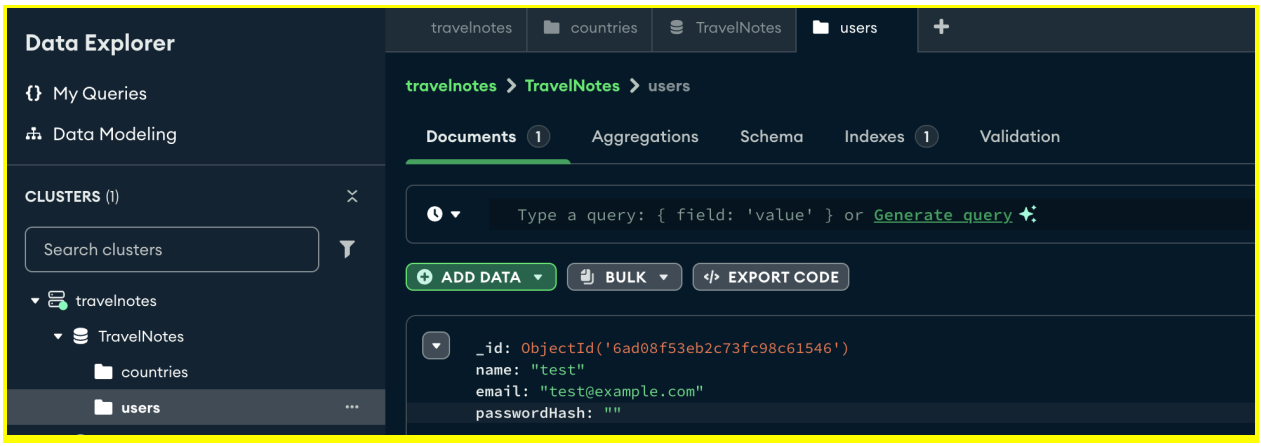
- *Mongoose sits between Express and MongoDB*
- *Cities, places and notes are embedded as subdocuments*
- *Validation lives in the schema: `required`, `enum` on `status`, and `min`/`max` on `rating` and the coordinates.*
- *Coordinates are stored as plain `lat`/`lng`*
- *The connection string lives in `.env` (ignored in git)*


Data Explorer
My Queries
Data Modeling
CLUSTERS (1)
Search clusters
travelnotes
TravelNotes
countries
users
admin
local
oplog.rs
sample_mflix
comments
embedded_movies
movies
sessions
theaters
users
test
travelnotes
countries
TravelNotes
users
travelnotes > TravelNotes > countries
Documents 2
Aggregations
Schema
Indexes 1
Validation
Type a query: { field: 'value' } or Generate query
ADD DATA
BULK
EXPORT CODE
_id: ObjectId('6ad08f11eb2c73fc98c6153f')
name: "Ireland"
cities: Array (2)
_id: ObjectId('6ad08f11eb2c73fc98c61540')
name: "Portugal"
cities: Array (1)
0 : Object (2)
name: "Lisbon"
places: Array (1)
0 : Object (5)
name: "Belém Tower"
category: "Landmark"
status: "Planned"
coords: Object (2)
lat: 38.6916
lng: -9.216
notes: Array (empty)

Some text


Data Explorer
My Queries
Data Modeling
CLUSTERS (1)
Search clusters
travelnotes
TravelNotes
countries
users
admin
local
oplog.rs
sample_mflix
comments
embedded_movies
movies
sessions
theaters
users
test
travelnotes
countries
TravelNotes
users
travelnotes > TravelNotes > countries
Documents 2
Aggregations
Schema
Indexes 1
Validation
Type a query: { field: 'value' } or Generate query
ADD DATA
BULK
EXPORT CODE
_id: ObjectId('6ad08f11eb2c73fc98c6153f')
name: "Ireland"
cities: Array (2)
0 : Object (2)
name: "Tralee"
places: Array (2)
0 : Object (5)
name: "Blennerville Windmill"
category: "Landmark"
status: "Visited"
coords: Object (2)
lat: 52.2645
lng: -9.7196
notes: Array (1)
0 : Object (4)
title: "Windy day at the windmill"
rating: 4
text: "Great little museum, windy day"
dateVisited: ISODate('2026-08-15T00:00:00.000+00:00') August 15, 2026 at 12:00:00 AM UTC
1 : Object (5)
name: "Siamsa Tire"
category: "Theatre"
status: "Planned"
coords: Object (2)
lat: 52.2718
lng: -9.6996
notes: Array (empty)

Some text

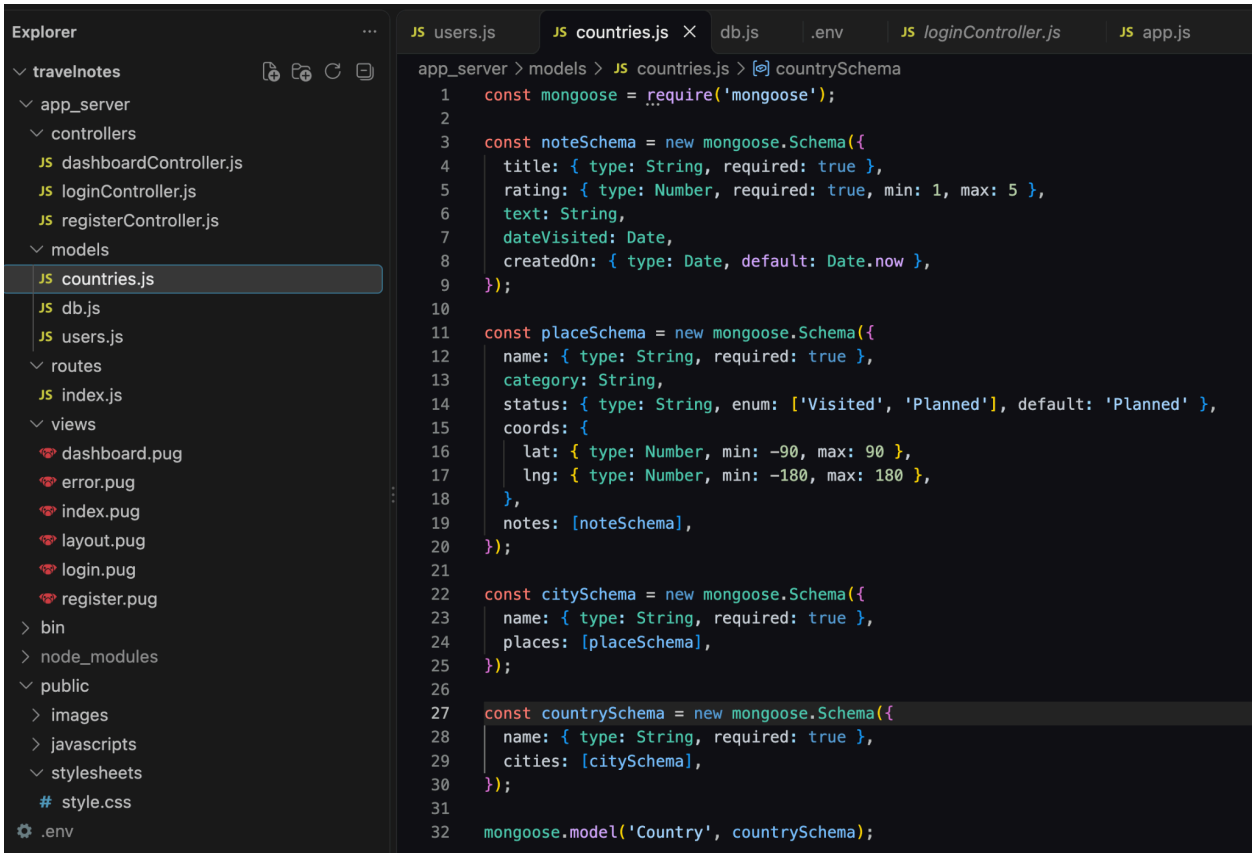

Data Explorer
My Queries
Data Modeling
CLUSTERS (1)
Search clusters
travelnotes
TravelNotes
countries
users
travelnotes
countries
TravelNotes
users
travelnotes > TravelNotes > users
Documents 1
Aggregations
Schema
Indexes 1
Validation
Type a query: { field: 'value' } or Generate query
ADD DATA
BULK
EXPORT CODE
_id: ObjectId('6ad08f53eb2c73fc98c61546')
name: "test"
email: "test@example.com"
passwordHash: ""

```
Daniels-MacBook-Air:travelnotes dk$ npm run dev

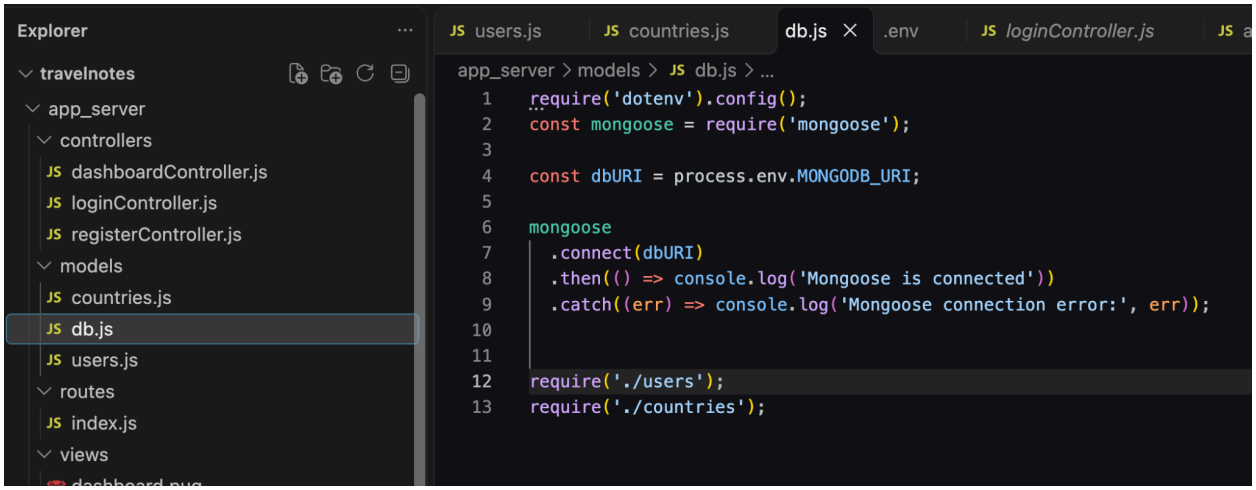
> travelnotes@0.0.0 dev
> nodemon ./bin/www

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node ./bin/www`
◇ injected env (1) from .env
Mongoose is connected
```

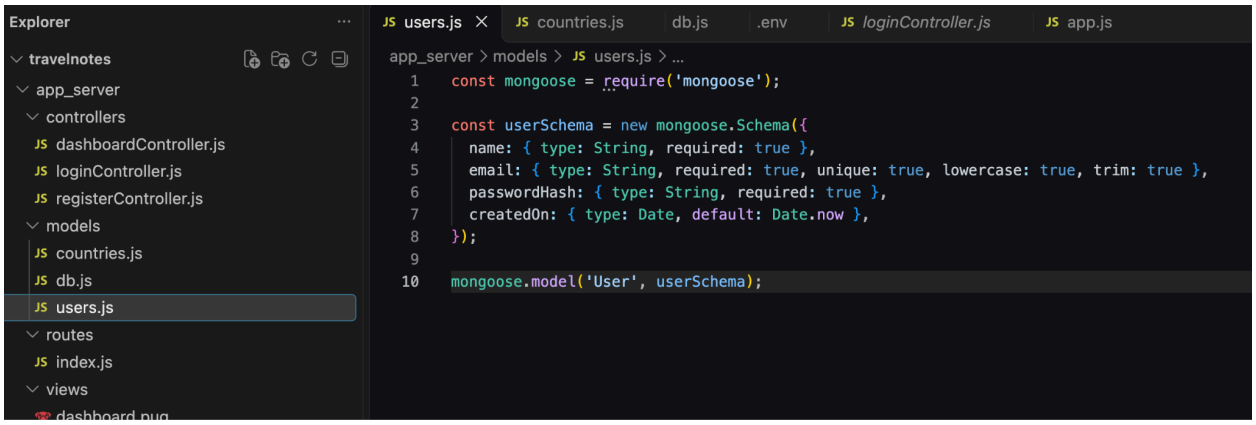
Sample text



Sample text



sample text



Sample text